

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Akhir
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Pengantar Mikrokontroler

Rozaq Nafi'ul Hafidz - 5024241072

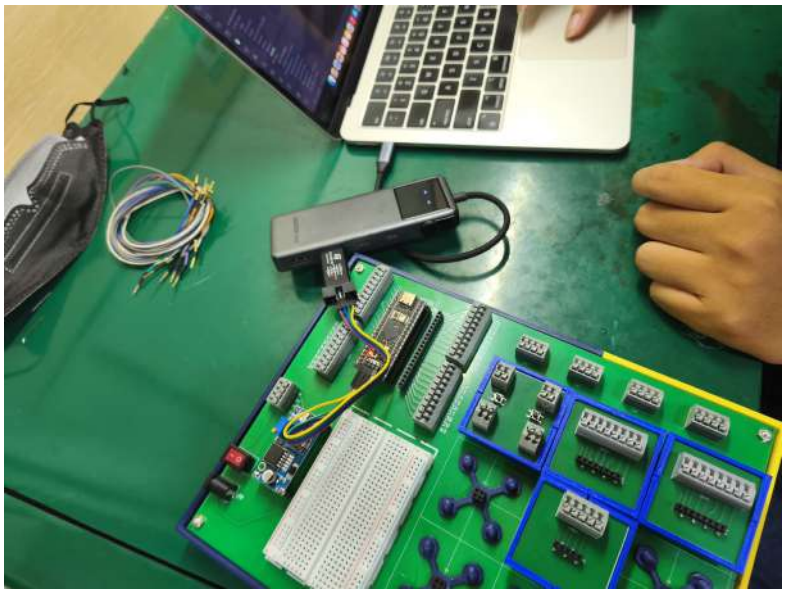
2026

1 Langkah-Langkah Percobaan

1.1 Persiapan Alat dan Lingkungan

Peralatan yang digunakan meliputi laptop, trainer praktikum, STM32 Blackpill, ST-Link V2, ESP32, breadboard, kabel USB, jumper, modul tombol, resistor, dan LDR.Untuk softwarenya, digunakan VSCode dan PlatformIO pada lingkungan desktop Linux serta macOS. Dokumentasi tambahan memperlihatkan port `/dev/cu.usbserial-140` dengan komunikasi 115200 baud.

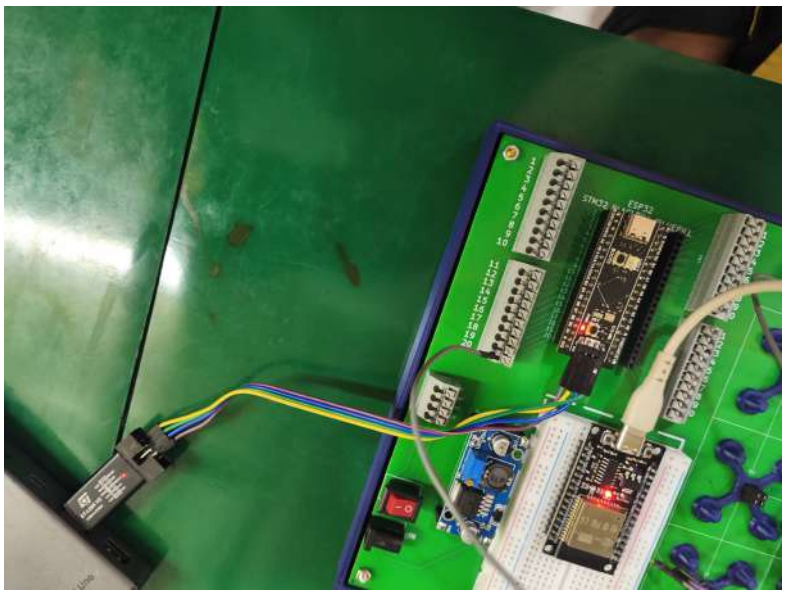
1. Menyiapkan trainer serta memeriksa board, catu daya, ground, jumper, dan kabel USB yang diperlukan.
2. Membuka proyek pada PlatformIO. Proyek ESP32 pada tangkapan layar bernama `percobaan2`; nama proyek ini tetap muncul pada tahapan pembacaan tombol.
3. Menyesuaikan board, framework, dan kode sumber sebelum menjalankan build/upload. Catu daya diputus sebelum perubahan wiring.



Gambar 1: Perangkat praktikum dan trainer yang digunakan.

1.2 Percobaan 1: Blink STM32 Blackpill

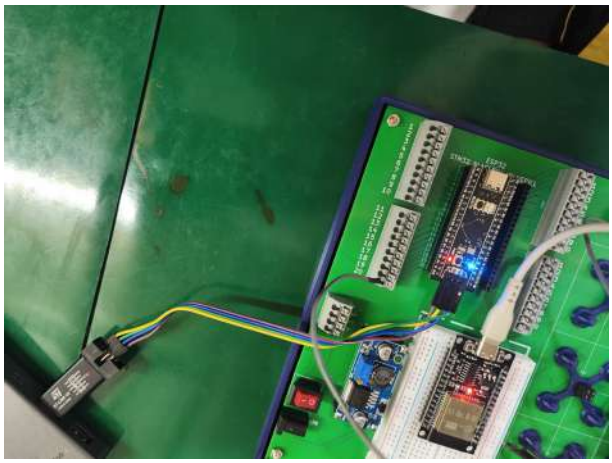
1. Menyiapkan konfigurasi acuan PlatformIO: `ststm32`, board `blackpill_f401cc`, framework Arduino, dan protokol upload ST-Link.
2. Menghubungkan programmer ST-Link dengan STM32 melalui SWDIO, SWCLK, ground, dan sambungan tegangan target yang sesuai.



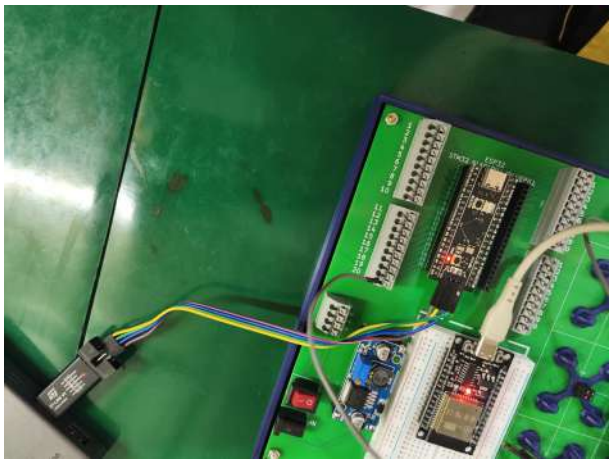
Gambar 2: Dokumentasi STM32 Blackpill, ST-Link V2, serta jumper pemrograman pada trainer.

3. Menggunakan program acuan Blink PC13 yang bergantian memberikan LOW dan HIGH, masing-masing selama 500 ms, kemudian melakukan build dan upload sesuai prosedur modul.

4. Mendokumentasikan dua keadaan LED STM32. Foto memperlihatkan LED biru menyala pada satu keadaan dan tidak menyala pada keadaan lain, sedangkan indikator merah tetap terlihat. Pengamatan menampilkan pergantian keadaan LED sesuai fungsi keluaran digital.

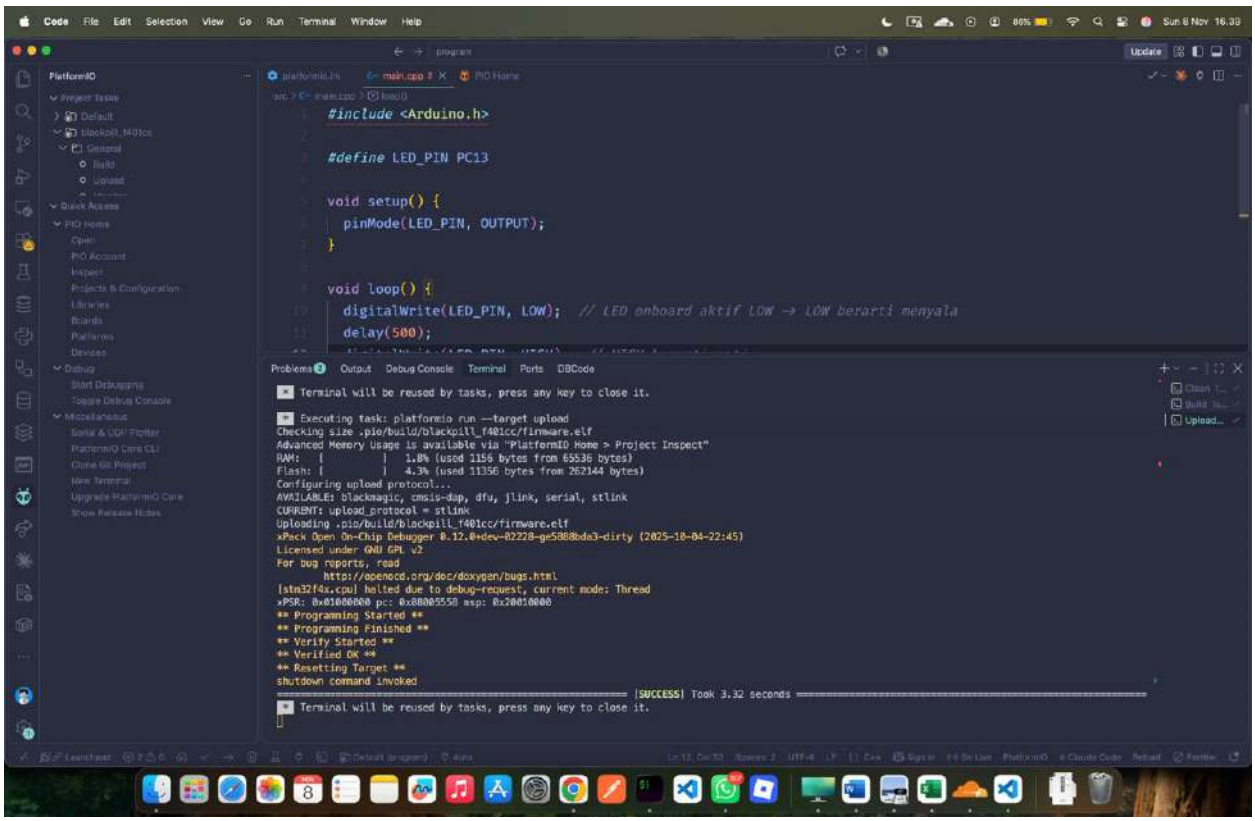


LED biru menyala.



LED biru tidak menyala.

Gambar 3: Dua keadaan LED pada STM32 Blackpill.

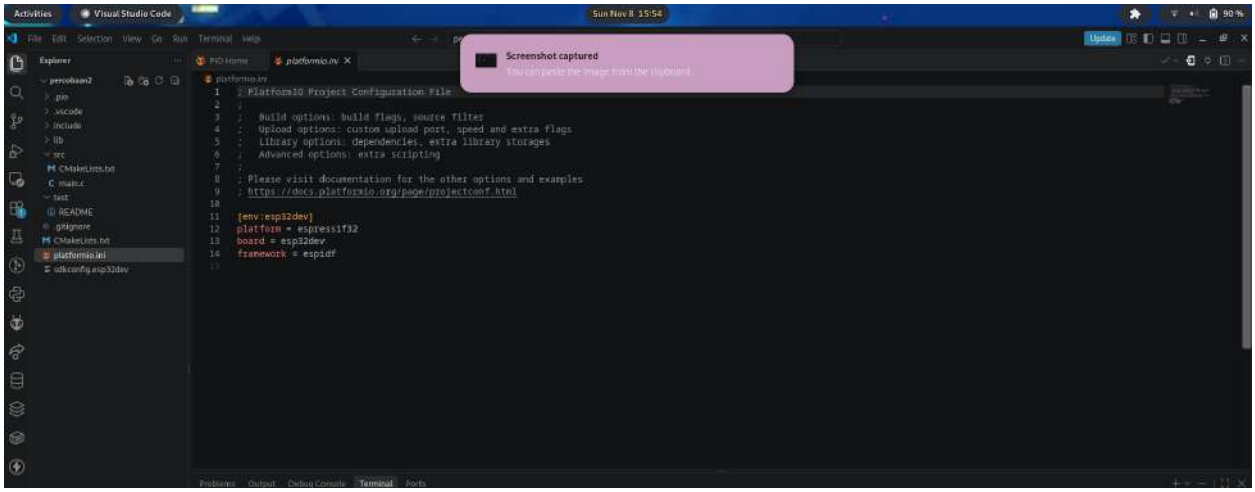


Gambar 4: Kode Blink STM32 PC13 dan upload melalui ST-Link berstatus SUCCESS.

Dokumentasi tambahan memperlihatkan target `blackpill_f401cc`, kode Arduino dengan `LED_PIN PC13`, dan `upload_protocol = stlink`. Log menunjukkan `Programming Finished`, `Verified OK`, reset target, lalu `[SUCCESS]` dengan waktu proses 3,32 s. RAM tercatat 1.156 byte dari 65.536 byte (1,8%) dan Flash 11.356 byte dari 262.144 byte (4,3%). Waktu tersebut adalah durasi proses upload, bukan periode LED.

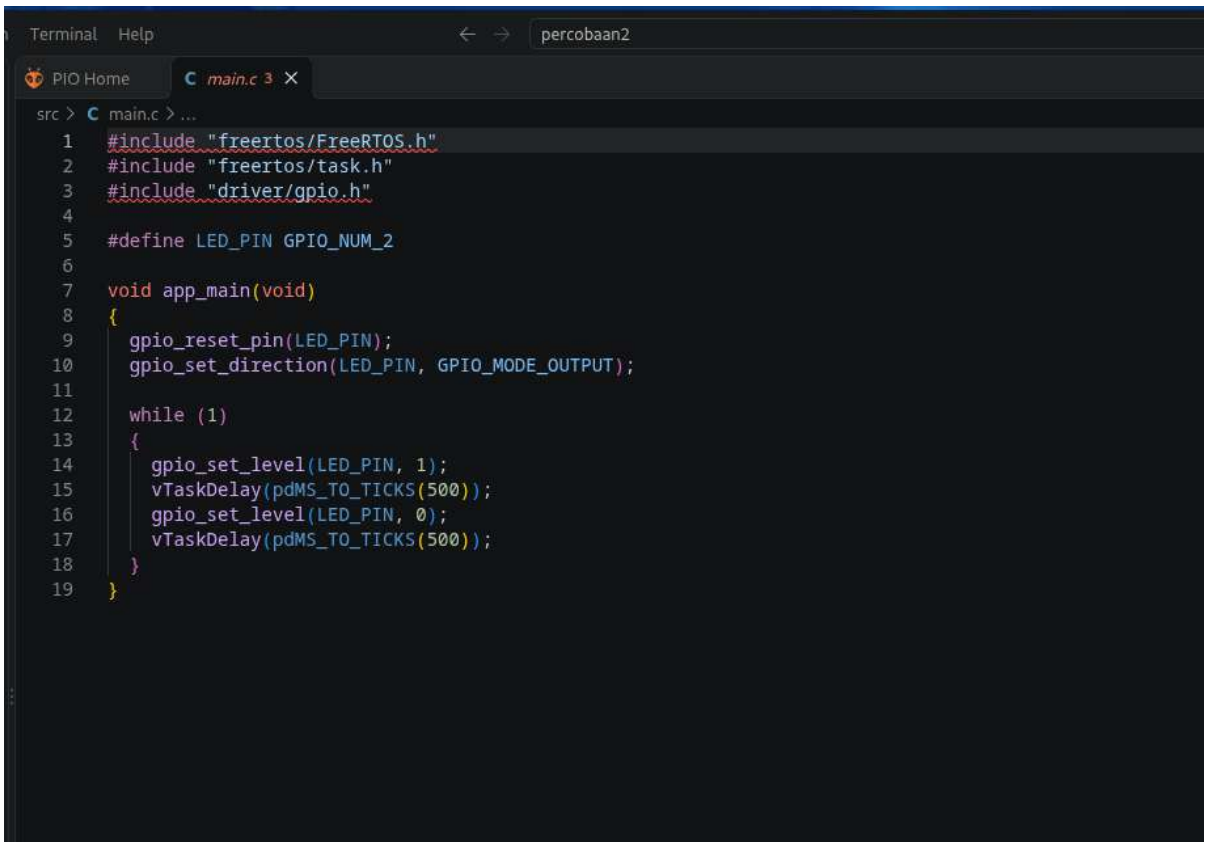
1.3 Percobaan 2: Blink ESP32 dengan ESP-IDF

1. Memilih `platform = espressif32`, `board = esp32dev`, dan `framework = espidf`. Berkas sumber awal berada pada `src/main.c`.



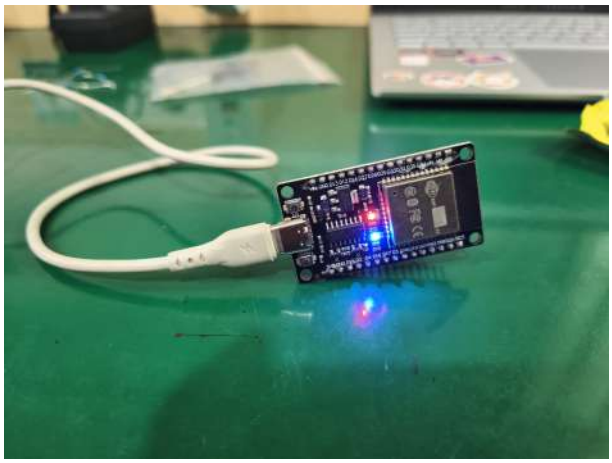
Gambar 5: Konfigurasi ESP32 dengan framework ESP-IDF.

2. Menulis `app_main()` yang mengatur GPIO2 sebagai output. Program memberi level 1 selama 500 ms dan level 0 selama 500 ms menggunakan API GPIO serta FreeRTOS.

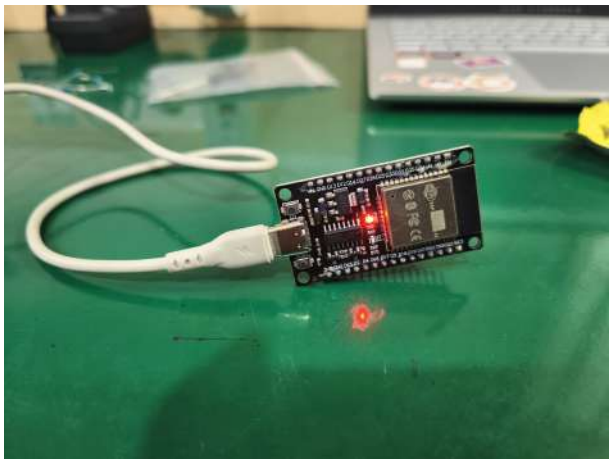


Gambar 6: Kode Blink ESP-IDF pada GPIO2 dengan jeda 500 ms per keadaan.

3. Menghubungkan ESP32 ke komputer melalui USB dan mendokumentasikan keadaan LED pada board.



LED biru menyala.



LED biru tidak menyala.

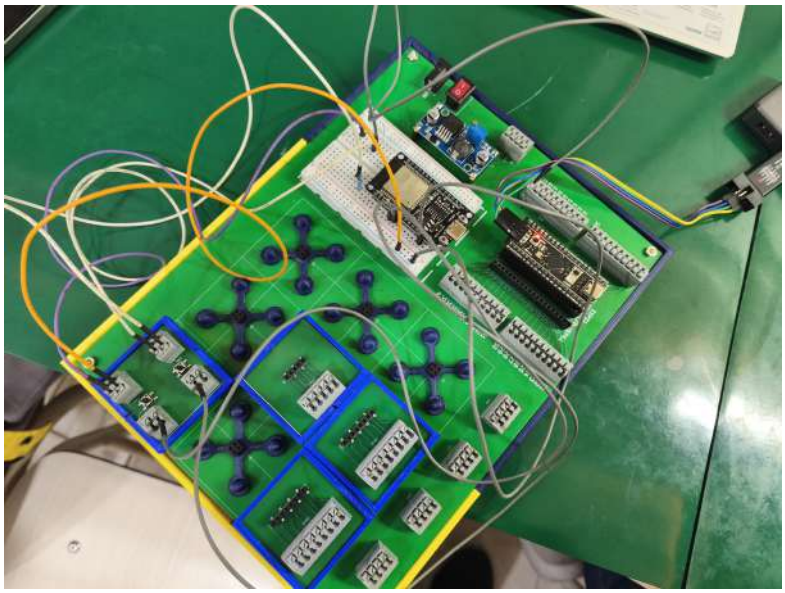
Gambar 7: Dua keadaan LED ESP32; indikator merah tetap terlihat.

Program ESP-IDF mengendalikan GPIO2 melalui `gpio_set_level()` dan menggunakan `vTaskDelay()` untuk mengatur jeda setiap keadaan. Urutan level 1 dan 0 membentuk pola nyala dan padam yang berulang.

1.4 Percobaan 3: Pull-up dan Pull-down Eksternal serta Internal

1. Menggunakan board `esp32dev` dengan framework Arduino. Mengatur GPIO32 untuk tombol pull-up dan GPIO33 untuk tombol pull-down.

2. Merangkai tombol dan resistor eksternal sesuai modul: resistor pull-up ke 3,3 V serta tombol ke ground pada GPIO32; resistor pull-down ke ground serta tombol ke 3,3 V pada GPIO33. Kedua GPIO diatur sebagai INPUT.



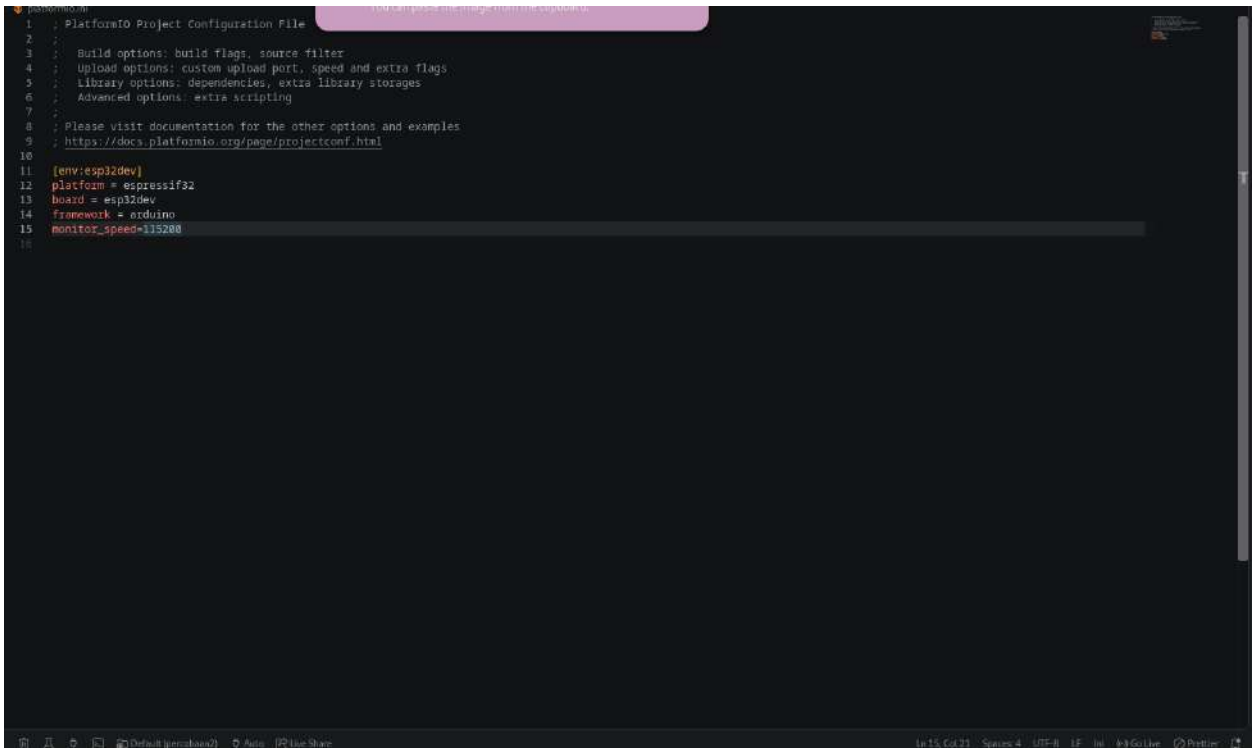
Gambar 8: Dokumentasi rangkaian tombol dan ESP32 pada trainer saat pengujian eksternal.



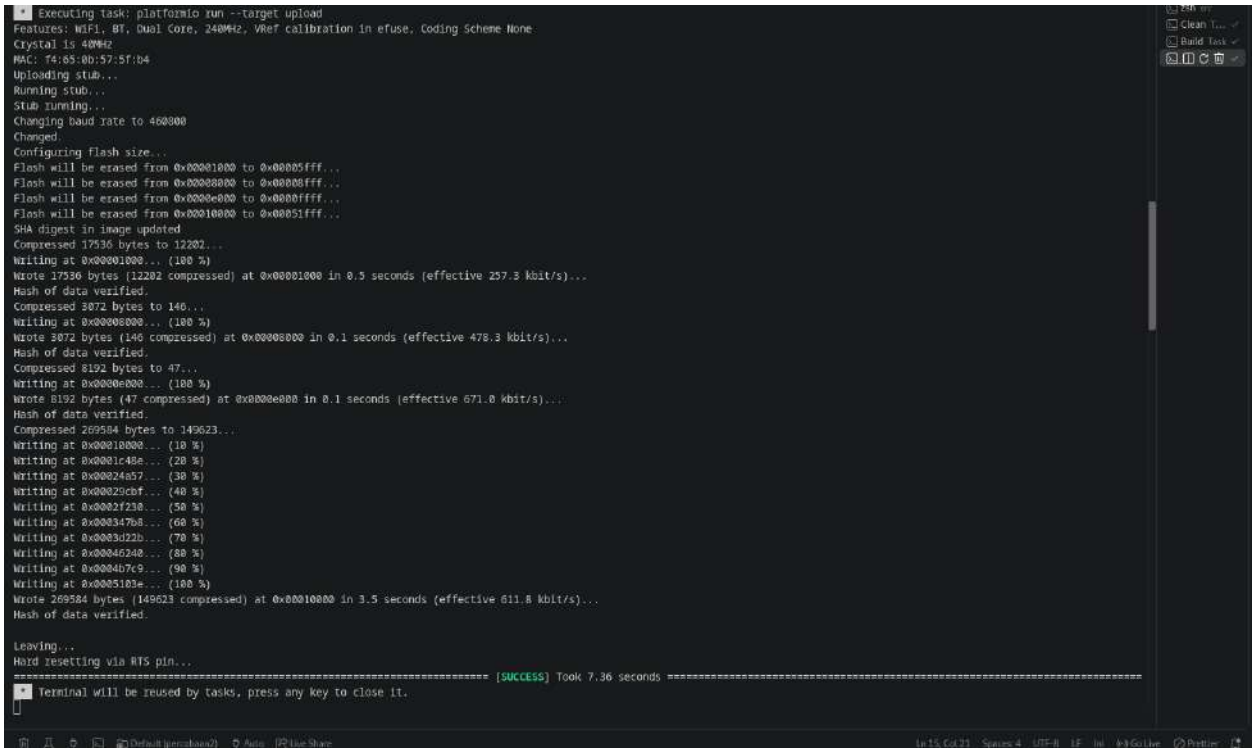
Gambar 9: Kode pembacaan tombol eksternal pada GPIO32 dan GPIO33.

3. Melakukan build dan upload kode eksternal. Log menunjukkan `[SUCCESS]` untuk build dengan waktu 5,13 s dan upload dengan waktu 7,36 s. Ukuran RAM tercatat 21.464 byte (6,6%) dan Flash 269.217 byte (20,5%) pada build yang terdokumentasi.

4. Menyesuaikan `monitor_speed=115200` dengan `Serial.begin(115200)`, lalu membuka Serial Monitor. Waktu pada log merupakan waktu proses build/upload, bukan waktu respons tombol.

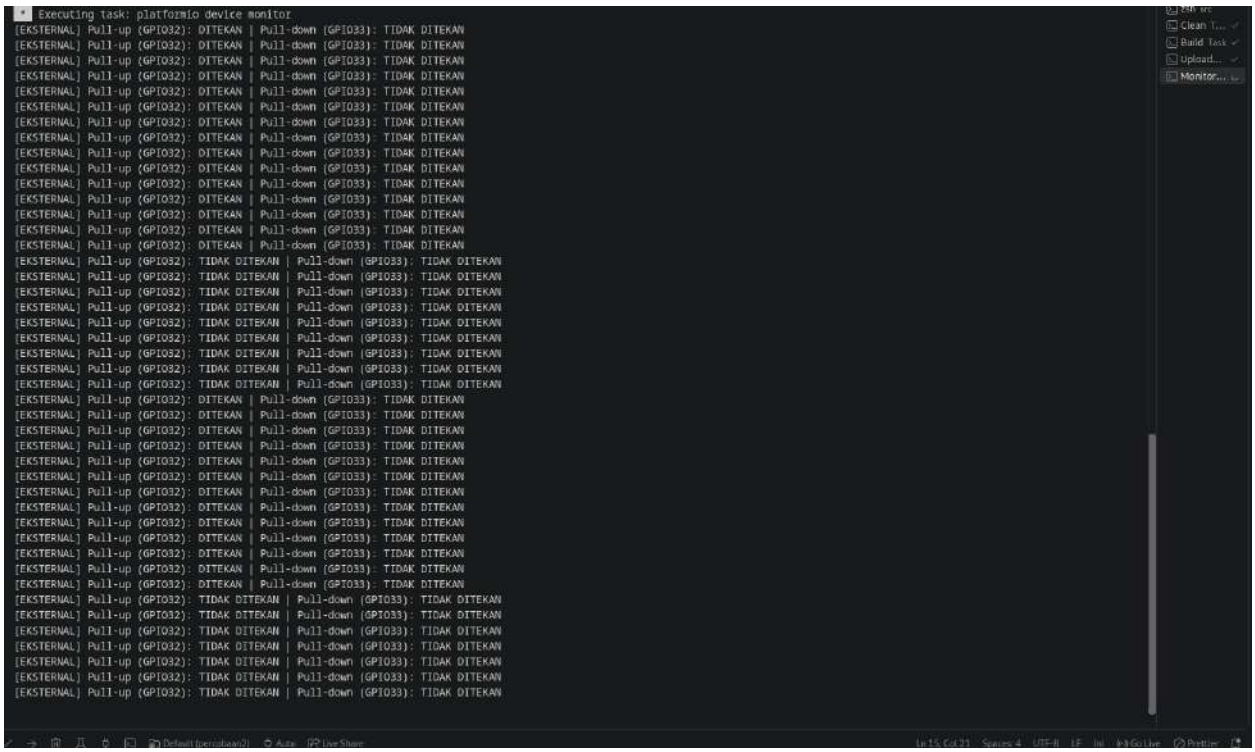


Gambar 10: Konfigurasi monitor serial 115200 baud untuk ESP32 Arduino.



Gambar 11: Upload program ESP32 pada rangkaian pengujian eksternal berstatus SUCCESS.

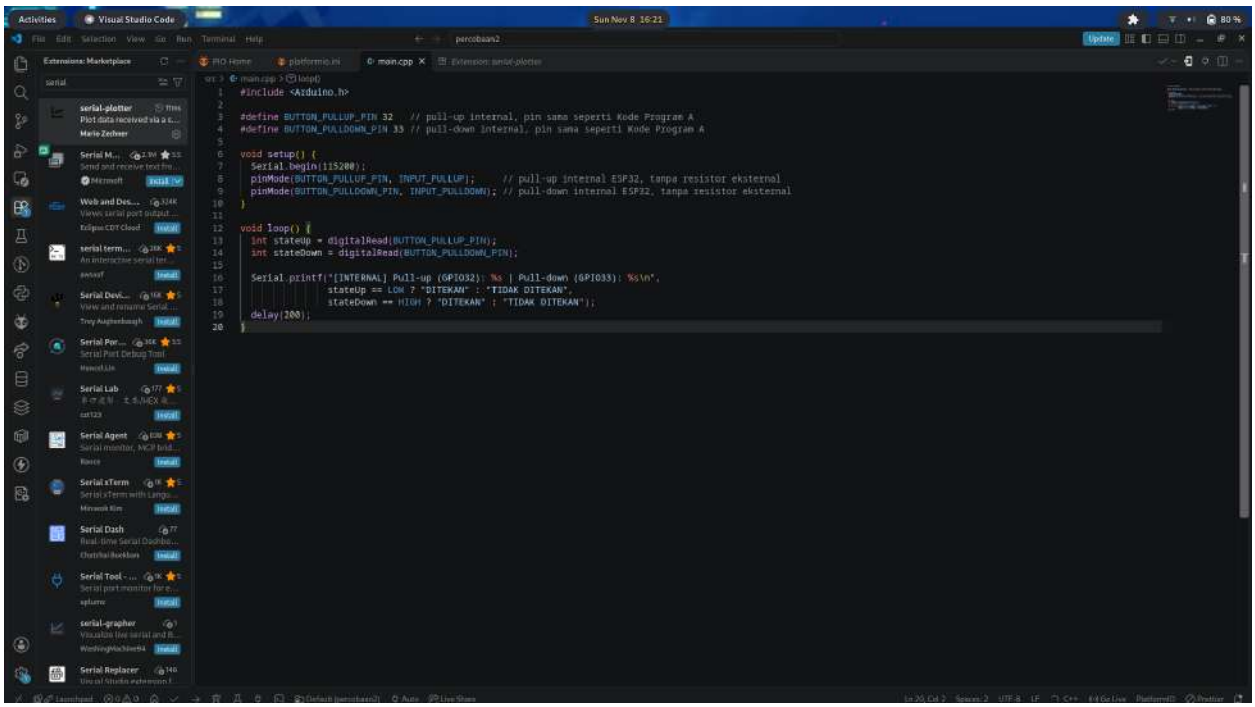
5. Merekam keluaran tombol. Pada cuplikan yang tersedia, pesan GPIO32 berubah antara DITEKAN dan TIDAK DITEKAN, sedangkan GPIO33 seluruhnya TIDAK DITEKAN. Dengan logika kode, keadaan itu menunjukkan GPIO32 berganti LOW/HIGH dan GPIO33 tetap LOW.



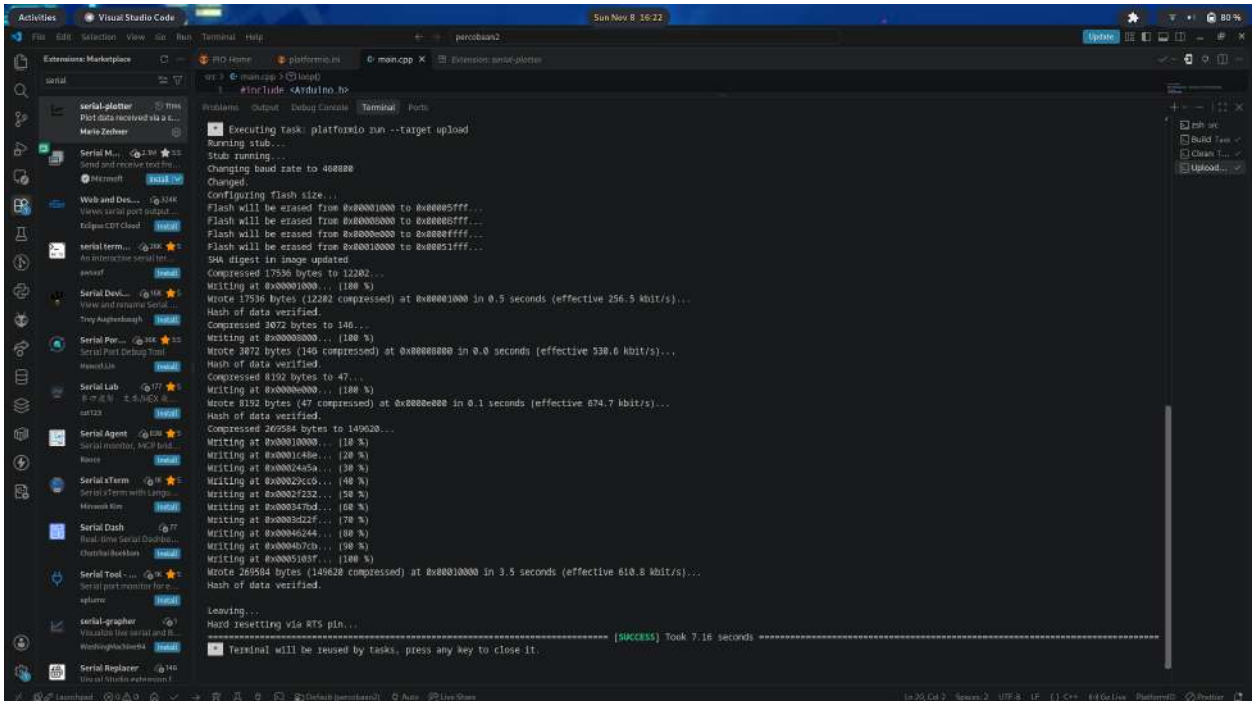
Gambar 12: Keluaran Serial Monitor pada konfigurasi eksternal.

6. Mengubah rangkaian menjadi konfigurasi internal sesuai modul dan menggunakan `INPUT_PULLUP` pada GPIO32 serta `INPUT_PULLDOWN` pada GPIO33. Pengaturan ini terlihat pada kode yang didokumentasikan.

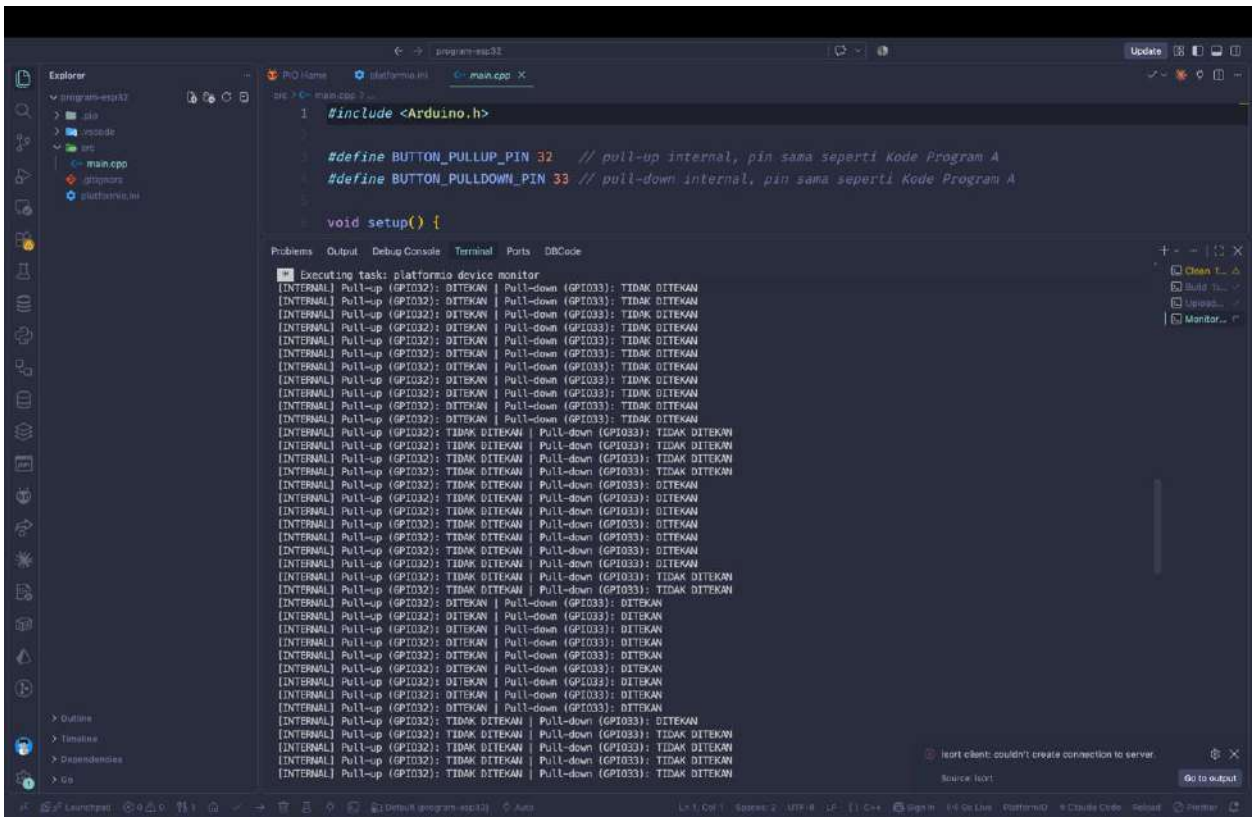
7. Melakukan build/upload kode internal. Log upload menunjukkan `[SUCCESS]` dengan waktu proses 7,16 s. Dokumentasi tambahan pada monitor internal memperlihatkan seluruh kombinasi pesan: kedua tombol tidak ditekan, hanya pull-up ditekan, hanya pull-down ditekan, dan keduanya ditekan.



Gambar 13: Kode konfigurasi pull-up dan pull-down internal.



Gambar 14: Upload tahap internal berstatus SUCCESS.

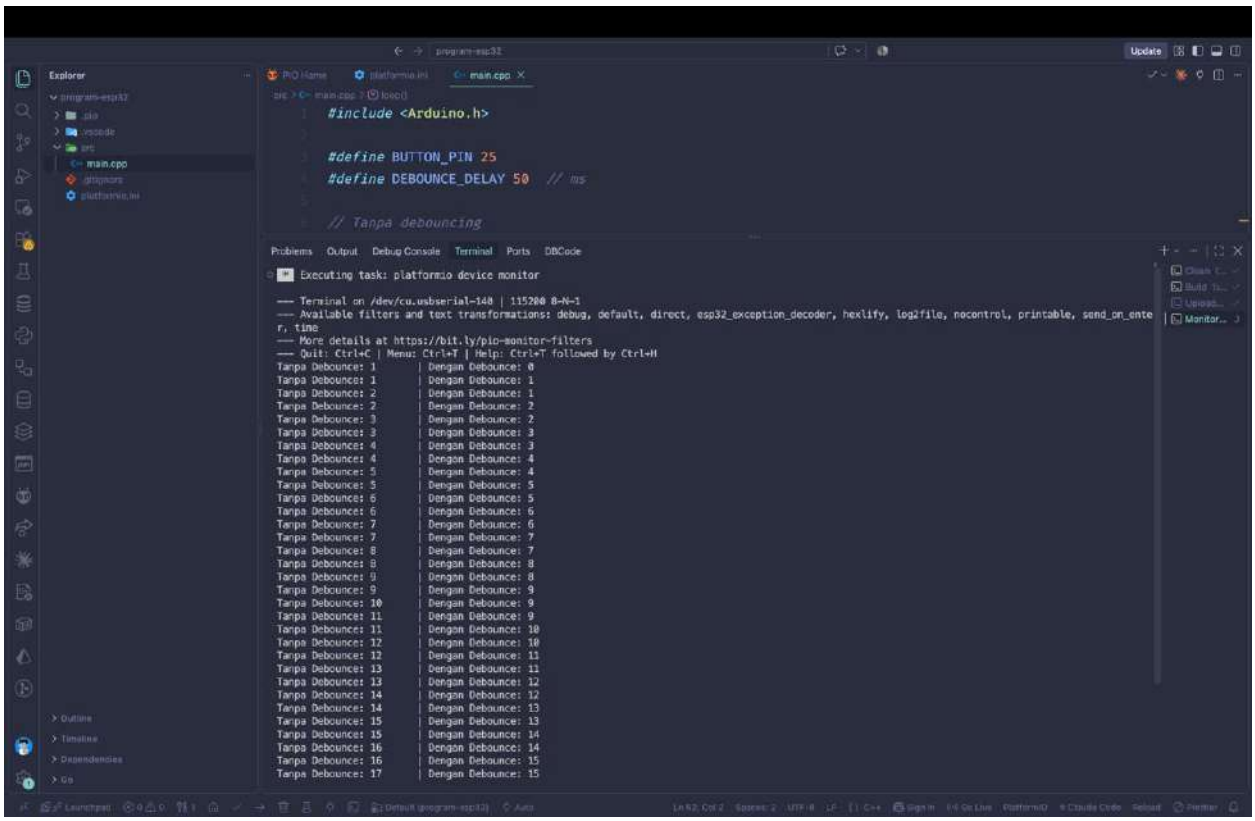


Gambar 15: Serial Monitor internal memperlihatkan empat kombinasi keadaan tombol.

Pengujian internal menampilkan pembacaan kedua tombol secara terpisah maupun bersamaan. Hasilnya memperlihatkan hubungan yang konsisten antara konfigurasi pull-up/pull-down dan pesan keadaan tombol.

1.5 Percobaan 4: Debouncing Tombol

1. Menghubungkan tombol ke GPIO25 dengan konfigurasi pull-up sesuai modul. Kode pada dokumentasi menunjukkan `BUTTON_PIN 25`.
2. Menjalankan program penghitung tanpa dan dengan debounce. Nilai `DEBOUNCE_DELAY` yang terlihat adalah 50 ms.
3. Membuka Serial Monitor pada `/dev/cu.usbserial-140`, 115200 baud, 8-N-1, lalu merekam perubahan kedua penghitung.
4. Membandingkan beberapa pasangan nilai yang tercatat, termasuk 1/0 lalu 1/1, 11/9, 12/10, dan 17/15 untuk counter tanpa/dengan debounce.

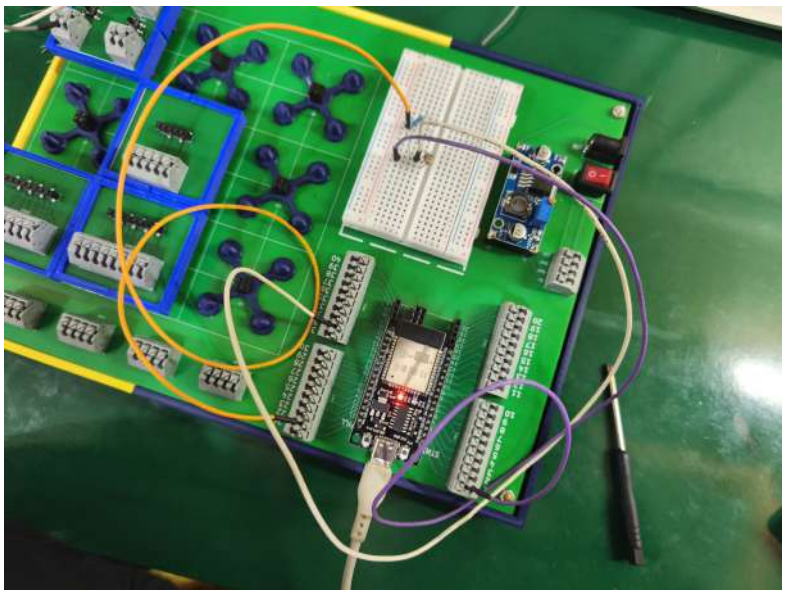


Gambar 16: Kode GPIO25, interval debounce 50 ms, dan keluaran kedua penghitung.

Pada akhir cuplikan, penghitung tanpa debounce bernilai 17 dan penghitung dengan debounce bernilai 15. Penyaring waktu membuat pembaruan keadaan stabil berbeda dari pendeteksian transisi mentah.

1.6 Percobaan 5: Pembacaan ADC dengan LDR

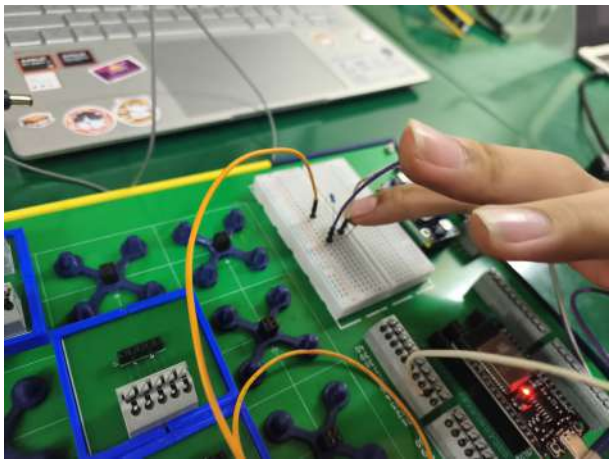
1. Menyiapkan pembagi tegangan LDR dan resistor pada breadboard. Menurut skema modul, LDR berada pada sisi 3,3 V, resistor 10 kΩ pada sisi ground, dan titik tengah dibaca melalui GPIO34.



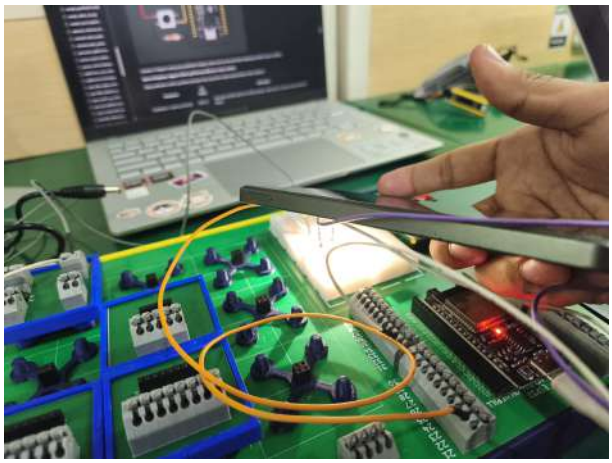
Gambar 17: Rangkaian LDR dan resistor pada breadboard yang terhubung ke ESP32.

2. Menggunakan program acuan `analogRead(34)` dan konversi `raw * (3.3 / 4095.0)`, kemudian membuka Serial Monitor untuk mencatat data.

3. Memberikan kondisi cahaya berbeda. Foto memperlihatkan komponen pada breadboard ditutupi tangan serta disinari lampu telepon genggam.



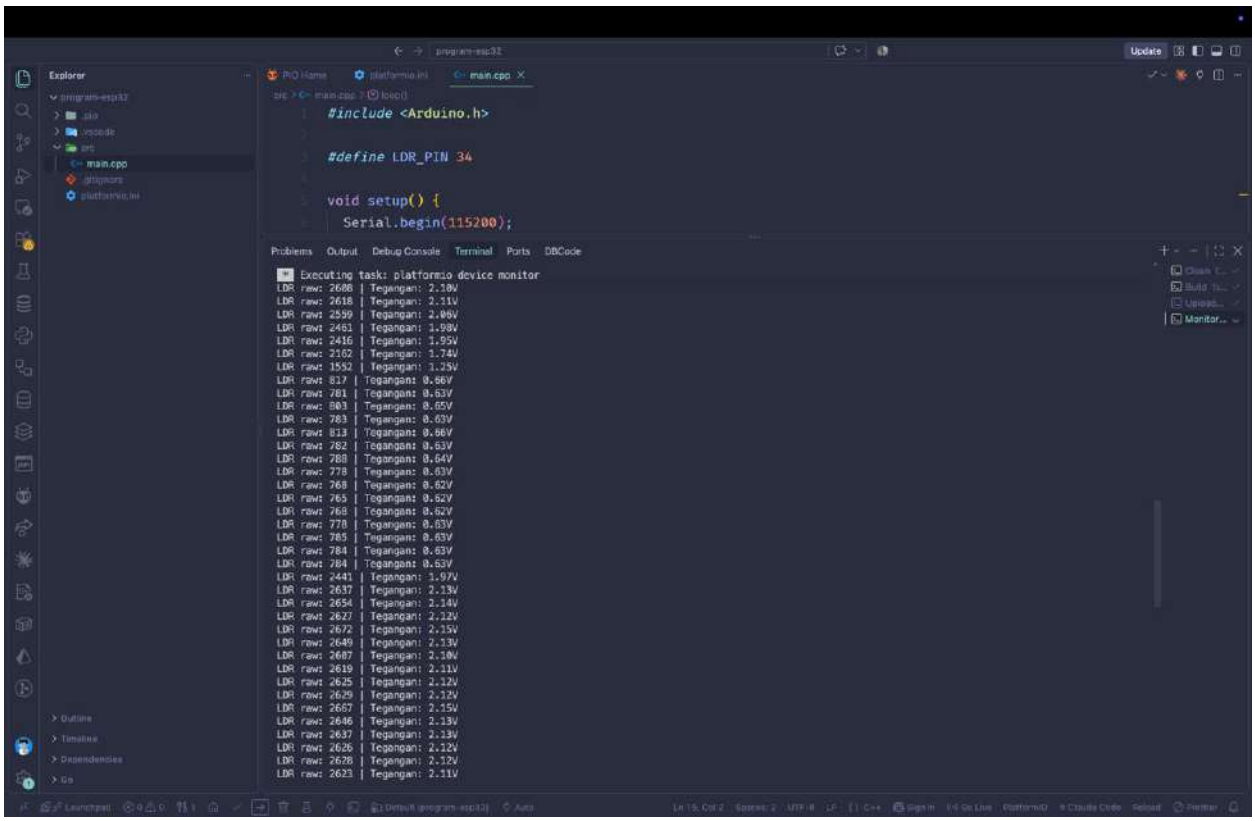
LDR ditutupi tangan.



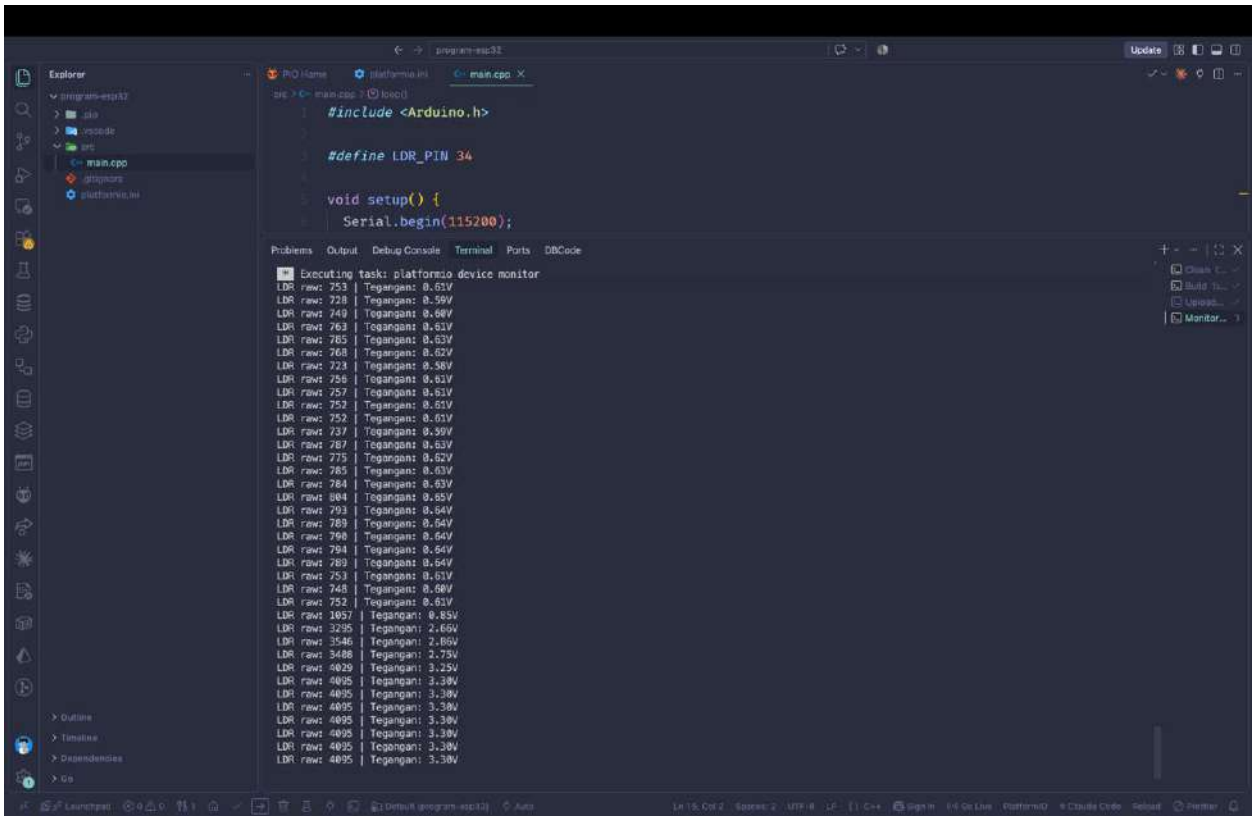
LDR disinari lampu telepon.

Gambar 18: Dokumentasi perubahan kondisi pencahayaan pada pengujian LDR.

4. Mendokumentasikan Serial Monitor. Kode yang terlihat memakai `LDR_PIN 34` dan `Serial.begin(115200)`. Cuplikan menunjukkan pembacaan sekitar 723–817 pada rentang rendah, sekitar 2.600 pada rentang menengah, dan mencapai 4095 pada rentang tertinggi.



Gambar 19: Pembacaan LDR pada GPIO34: perubahan dari sekitar 2.600 ke sekitar 780 dan kembali meningkat.



Gambar 20: Pembacaan LDR dari rentang rendah sampai batas kode ADC 4095.

Pembacaan dianalisis berdasarkan kelompok nilai rendah, menengah, dan mendekati batas atas ADC. Pengelompokan tersebut memudahkan perbandingan perubahan kode digital dan tegangan

hasil konversi program.

2 Analisis Hasil Percobaan

2.1 Blink STM32 dan ESP32

LED biru pada STM32 dan ESP32 memperlihatkan keadaan menyala dan padam, sedangkan indikator daya merah tetap menyala. Perubahan ini menunjukkan pemanfaatan GPIO sebagai keluaran digital. Pada STM32, pemrograman melalui ST-Link berakhir dengan `SUCCESS` dan `Verified OK`, yang menunjukkan firmware telah dituliskan serta diverifikasi.

Tabel 1: Perbandingan pengendalian LED pada kedua platform.

Aspek	STM32 Arduino	ESP32 ESP-IDF
Pin program	PC13	GPIO2
Struktur program	setup() dan loop()	app_main() dan perulangan while
Pengendalian GPIO	digitalWrite()	gpio_set_level()
Jeda program	delay(500)	vTaskDelay() selama 500 ms
Keadaan LED pada foto	Menyala dan padam	Menyala dan padam

LED STM32 aktif LOW sehingga penulisan LOW menyalakannya. Pada kode ESP32, perubahan level 1 dan 0 dijalankan bergantian dengan jeda 500 ms. Berdasarkan pengaturan program, periode rancangan adalah $T = 500 + 500 = 1000$ ms atau 1 s, dengan frekuensi 1 Hz dan duty cycle 50%. Nilai ini menjelaskan pola waktu yang dibentuk kode. Durasi upload STM32 sebesar 3,32 s berkaitan dengan pemrograman perangkat dan berbeda dari periode nyala LED.

Perbedaan utama kedua framework terdapat pada tingkat abstraksi dan pengelolaan waktu. Arduino menyediakan fungsi yang ringkas untuk inisialisasi serta perulangan aplikasi. ESP-IDF menggunakan API GPIO dan fungsi FreeRTOS, sehingga penundaan dilakukan pada task pemanggil dan task lain dapat dijadwalkan selama waktu tunggu. Kedua pendekatan menghasilkan kendali keluaran digital melalui struktur program yang berbeda.

2.2 Pull-up dan Pull-down

Bukti eksternal memuat kode yang mengartikan GPIO32 LOW sebagai DITEKAN dan HIGH sebagai TIDAK DITEKAN. Pada GPIO33, HIGH diartikan DITEKAN dan LOW diartikan TIDAK DITEKAN. Keluaran yang dapat dibaca dari tangkapan layar adalah:

Listing 1: Cuplikan jenis pesan yang terlihat pada monitor eksternal.

```
[EKSTERNAL] Pull-up (GPIO32): DITEKAN | Pull-down (GPIO33): TIDAK DITEKAN
[EKSTERNAL] Pull-up (GPIO32): TIDAK DITEKAN | Pull-down (GPIO33): TIDAK DITEKAN
```

Pesan berulang menunjukkan GPIO32 sempat berada pada dua keadaan logika, sedangkan GPIO33 pada bagian log yang tersedia tetap terbaca LOW. Level pada tabel berikut diturunkan dari pesan dan percabangan pada kode; bukan hasil pengukuran tegangan dengan alat ukur.

Tabel 2: Keadaan yang benar-benar terdokumentasi pada monitor eksternal.

Pesan GPIO32/GPIO33	GPIO32	GPIO33	Interpretasi kode
DITEKAN / TIDAK DITEKAN	LOW	LOW	Pull-up aktif; pull-down tidak aktif
TIDAK DITEKAN / TIDAK DITEKAN	HIGH	LOW	Kedua tombol berstatus tidak ditekan

Perubahan pembacaan GPIO32 dari HIGH ke LOW sesuai dengan logika pull-up aktif LOW. GPIO33 pada kedua cuplikan berada pada LOW, yaitu keadaan tidak aktif pada konfigurasi pull-down. Hasil ini memperlihatkan fungsi resistor dalam menetapkan level awal masukan dan perbedaan cara program mengartikan keadaan tombol.

Pada konfigurasi internal, kode mengaktifkan `INPUT_PULLUP` dan `INPUT_PULLDOWN`, upload berakhir SUCCESS, dan foto tambahan memperlihatkan keluaran Serial Monitor. Seluruh kombinasi keadaan yang diharapkan muncul. Level pada tabel diturunkan dari pesan program dan logika kode yang telah didokumentasikan.

Tabel 3: Hasil pembacaan tombol internal dari foto tambahan b.

Status menurut program	GPIO32	GPIO33	Perbandingan teori
Kedua tombol tidak ditekan	HIGH	LOW	Sesuai
Hanya pull-up ditekan	LOW	LOW	Sesuai
Hanya pull-down ditekan	HIGH	HIGH	Sesuai
Kedua tombol ditekan	LOW	HIGH	Sesuai

Konfigurasi internal menghasilkan empat kombinasi keadaan yang sesuai dengan logika masukan. Pada dua kombinasi yang juga terlihat pada pengujian eksternal, pembacaannya sejalan: GPIO32 berubah sesuai tombol pull-up dan GPIO33 berada pada level tidak aktif. Perbedaan eksternal dan internal terletak pada asal resistor penentu keadaan awal, sedangkan makna HIGH/LOW tetap mengikuti konfigurasi tombol.

Pengaturan monitor 115200 baud sesuai dengan inisialisasi serial pada kode. Durasi upload 7,36 s untuk program eksternal dan 7,16 s untuk program internal adalah waktu proses transfer firmware. Selisih tersebut tidak digunakan untuk menilai kecepatan respons tombol. Keandalan pembacaan dipengaruhi kesesuaian mode pin, jalur ground, dan pemilihan pasangan kaki pushbutton.

2.3 Debouncing

Dokumentasi tambahan memperlihatkan penggunaan GPIO25 dan interval debounce 50 ms. Counter tanpa debounce bertambah langsung ketika transisi mentah terdeteksi, sedangkan counter dengan debounce menerima perubahan setelah keadaan stabil. Beberapa pasangan nilai yang dapat dibaca disajikan berikut. Urutan pada tabel merupakan cuplikan dari satu tampilan, bukan seri pengujian terpisah.

Tabel 4: Cuplikan nilai counter pada interval debounce 50 ms.

Tanpa debounce	Dengan debounce	Selisih
1	0	1
1	1	0
10	9	1
11	9	2
12	10	2
17	15	2

Perubahan dari 1/0 menjadi 1/1 menunjukkan penghitung terfilter dapat tertunda dibandingkan penghitung mentah saat menunggu validasi kestabilan. Pada akhir cuplikan, counter mentah 17 dan counter terfilter 15 menunjukkan selisih dua hitungan. Hasil ini konsisten dengan adanya transisi yang tidak langsung diterima oleh penyaring. Namun, selisih sesaat tidak dapat langsung dinyatakan sebagai dua kesalahan akibat bouncing: penyaringan juga menunda penerimaan dan dapat menolak tekanan yang terlalu singkat.

Penghitungan berbasis keadaan stabil mengurangi penerimaan transisi yang terlalu singkat. Selisih akhir dua hitungan menggambarkan perbedaan antara event mentah dan event yang lolos validasi waktu. Interval 50 ms memberikan waktu bagi masukan untuk stabil sebelum perubahan diteruskan ke penghitung. Secara prinsip, interval yang lebih singkat meningkatkan kecepatan penerimaan perubahan, sedangkan interval yang lebih panjang menambah penyaringan tetapi dapat menolak tekanan singkat. Analisis hasil ini berfokus pada interval 50 ms yang digunakan dalam pengujian.

2.4 ADC dan LDR

Dokumentasi rangkaian dan perlakuan cahaya dilengkapi dengan dua cuplikan Serial Monitor. Kode pada gambar menunjukkan `LDR_PIN 34` dan serial 115200 baud. Pada foto d, nilai sempat berada sekitar 2.600, turun ke sekitar 780, lalu kembali ke sekitar 2.600. Pada foto e, nilai rendah diikuti kenaikan melalui kode 1057, 3295, 3546, dan 3408, kemudian mencapai 4095. Perubahan ini menunjukkan masukan analog direpresentasikan oleh banyak tingkat nilai, berbeda dengan masukan digital HIGH/LOW.

Tabel 5: Sampel ADC dan tegangan yang benar-benar terbaca pada foto tambahan.

Sumber	Kode ADC	Tegangan tampilan (V)	Keterangan
d	2608	2,10	Sebelum penurunan pada cuplikan
d	817	0,66	Menuju kelompok nilai rendah
d	765	0,62	Sampel pada kelompok rendah
d	2637	2,13	Setelah pembacaan kembali meningkat
e	723	0,58	Nilai terendah yang terlihat
e	1057	0,85	Kenaikan dari kelompok rendah
e	3295	2,66	Sampel pada rentang lebih tinggi
e	3546	2,86	Sampel pada rentang lebih tinggi
e	4092	3,30	Mendekati kode maksimum; tampilan dibulatkan
e	4095	3,30	Mencapai kode maksimum 12 bit

Nilai tegangan pada tabel adalah keluaran konversi program, bukan hasil multimeter. Perhitungan dari sampel yang terdokumentasi memberikan

$$V_{2608} = \frac{2608}{4095} \times 3{,}3 \approx 2{,}10\ \mathrm{V}, \qquad V_{723} = \frac{723}{4095} \times 3{,}3 \approx 0{,}58\ \mathrm{V}. \tag{1}$$

Hasil tersebut sesuai dengan angka pada Serial Monitor. Untuk kode 4092, estimasi sekitar 3,298 V dibulatkan menjadi 3,30 V; kode 4095 juga ditampilkan sebagai 3,30 V. Dengan demikian, dua kode yang berbeda dapat memiliki tegangan tampilan sama karena pembulatan dua angka desimal.

Rentang seluruh cuplikan yang terlihat adalah 723–4095, dengan selisih 3372 kode. Pembacaan 4095 menunjukkan batas atas kode telah tercapai sehingga ada indikasi saturasi pada kondisi tersebut. Penambahan tegangan atau cahaya setelah batas tercapai tidak lagi dapat dibedakan oleh kode

keluaran. Nilai 3,30 V pada tampilan tidak membuktikan tegangan fisik tepat 3,30 V, karena konversi memakai pendekatan linier dan karakteristik ADC aktual dipengaruhi atenuasi serta kalibrasi.

Pada pembagi tegangan dengan LDR di sisi suplai dan resistor tetap di sisi ground, peningkatan pencahayaan menurunkan hambatan LDR sehingga tegangan titik tengah meningkat. Foto percobaan memperlihatkan perlakuan menutup LDR dan menyinarinya, sementara data serial menunjukkan perubahan dari kelompok kode rendah menuju kelompok yang lebih tinggi. Analisis numerik menggunakan kelompok pembacaan tersebut untuk membandingkan respons ADC tanpa menyamakan kode digital dengan satuan intensitas cahaya.

Fluktuasi di sekitar kode 780 dan 2.600 menunjukkan pembacaan analog dapat berubah secara bertahap. Posisi tangan, arah sumber cahaya, kestabilan suplai, dan noise ADC dapat memengaruhi respons. Perhitungan tegangan tetap merupakan estimasi linier program. Pencapaian kode 4095 menunjukkan pentingnya memilih rentang pengukuran agar perubahan masukan tetap dapat dibedakan sebelum saturasi.

3 Hasil Tugas Modul

3.1 Rancangan Simulasi Matriks Tombol dan LED 3×3

Simulasi dibuat menggunakan Wokwi dengan ESP32 DevKit C V4, sembilan pushbutton, sembilan LED merah, serta sembilan resistor pembatas arus 220 Ω. Tombol disusun sebagai matriks tiga baris dan tiga kolom sehingga dapat dipindai menggunakan enam GPIO. LED disusun dalam pola 3×3, tetapi masing-masing dikendalikan langsung oleh satu GPIO. Dengan demikian, pemindaian baris/kolom diterapkan pada sisi tombol, sedangkan pembaruan LED menggunakan sembilan keluaran terpisah.

Baris tombol menggunakan GPIO16, GPIO17, dan GPIO18. Kolom menggunakan GPIO19, GPIO21, dan GPIO22 dengan `INPUT_PULLUP`. Anoda LED terhubung ke GPIO keluarannya, sedangkan jalur katoda dihubungkan ke ground melalui resistor 220 Ω. Tabel berikut menunjukkan pemetaan setiap tombol dan LED sesuai kode serta `diagram.json`.

Tabel 6: Pemetaan pin tombol dan LED pada simulasi.

Tombol	Posisi	GPIO baris	GPIO kolom	GPIO LED
T1	(1,1)	16	19	23
T2	(1,2)	16	21	25
T3	(1,3)	16	22	26
T4	(2,1)	17	19	27
T5	(2,2)	17	21	32
T6	(2,3)	17	22	33
T7	(3,1)	18	19	13
T8	(3,2)	18	21	14
T9	(3,3)	18	22	4

3.2 Pemindaian dan Debouncing Per Tombol

Program menginisialisasi baris sebagai INPUT, kolom sebagai INPUT_PULLUP, dan seluruh LED sebagai OUTPUT dalam keadaan LOW. Saat memindai, satu baris terlebih dahulu diberi nilai LOW lalu diubah menjadi OUTPUT. Program menunggu 10 mikrodetik agar jalur menetap, membaca ketiga

kolom, kemudian mengembalikan baris tersebut menjadi INPUT sebelum mengaktifkan baris berikutnya. Pengembalian ke INPUT membuat baris yang tidak dipilih berada pada impedansi tinggi, sehingga tidak dipaksa mengeluarkan level yang berlawanan dengan baris aktif.

Tombol dianggap ditekan ketika kolom pada baris aktif terbaca LOW. Fungsi `updateButton()` menyimpan pembacaan mentah pada `lastRaw`, keadaan stabil pada `stablePressed`, dan waktu perubahan pada `lastChange`. Ketiga array berukuran 3×3 sehingga setiap tombol mempunyai riwayat sendiri. Jika pembacaan berubah, waktu acuannya diperbarui. Keadaan stabil baru diterima setelah pembacaan bertahan minimal 35 ms dan berbeda dari keadaan stabil sebelumnya.

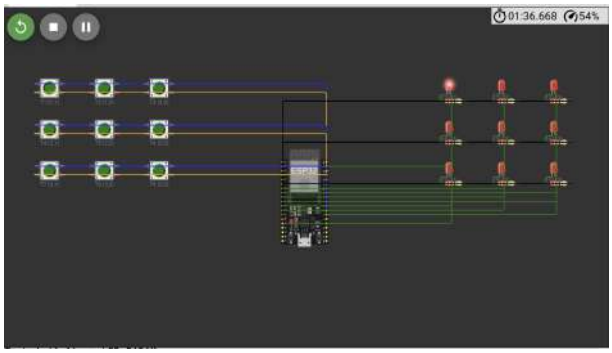
Setelah validasi, LED posisi terkait diberikan HIGH jika tombol ditekan dan LOW jika tombol dilepas. Perilaku ini mengikuti keadaan tombol stabil; LED tidak dibuat toggle untuk setiap penekanan. Program juga mengirim pesan posisi tombol dan status MENYALA/PADAM melalui serial 115200 baud. Setelah semua baris dipindai, terdapat jeda 1 ms sebelum siklus berikutnya. Pada berkas rangkaian, seluruh tombol diberi atribut `bounce = 1`, sedangkan penyaring pada program tetap diterapkan secara terpisah untuk setiap posisi.

3.3 Hasil Pengujian Sembilan Posisi

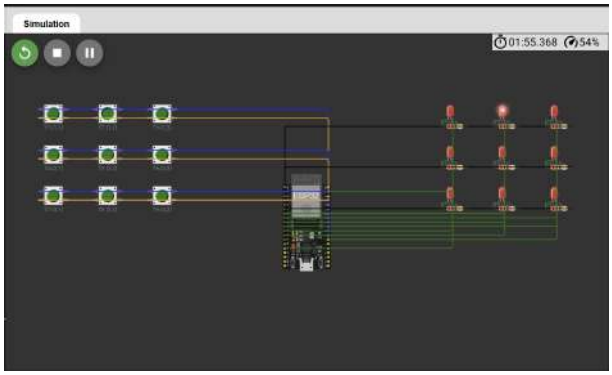
Sembilan tangkapan layar memperlihatkan LED L1 sampai L9 menyala bergantian pada susunan yang sesuai dengan posisi tombol. Pada setiap gambar, satu LED tujuan terlihat menyala dan LED lain padam. Pemetaan keluaran pada gambar sejalan dengan indeks baris/kolom pada array `ledPins`.

Tabel 7: Hasil visual dan Serial Monitor untuk setiap posisi tombol.

Tombol	Posisi	LED tujuan	Saat ditekan	Saat dilepas
T1	(1,1)	L1 (1,1)	MENYALA	PADAM
T2	(1,2)	L2 (1,2)	MENYALA	PADAM
T3	(1,3)	L3 (1,3)	MENYALA	PADAM
T4	(2,1)	L4 (2,1)	MENYALA	PADAM
T5	(2,2)	L5 (2,2)	MENYALA	PADAM
T6	(2,3)	L6 (2,3)	MENYALA	PADAM
T7	(3,1)	L7 (3,1)	MENYALA	PADAM
T8	(3,2)	L8 (3,2)	MENYALA	PADAM
T9	(3,3)	L9 (3,3)	MENYALA	PADAM

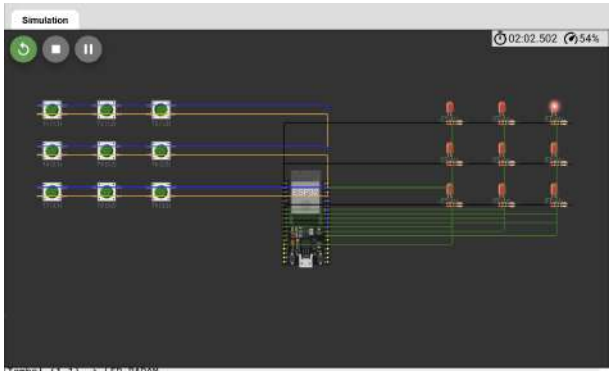


Pengujian T1 ke L1 (1,1).

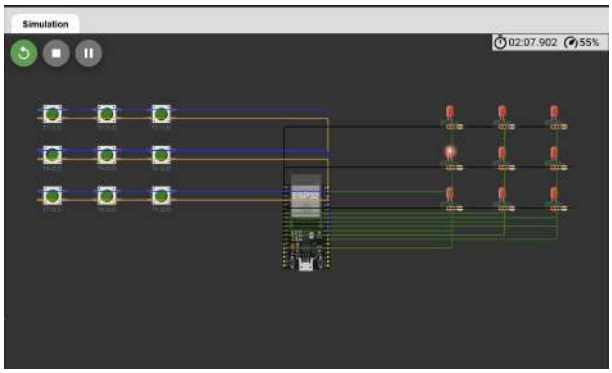


Pengujian T2 ke L2 (1,2).

Gambar 21: Hasil pengujian posisi (1,1) dan (1,2).

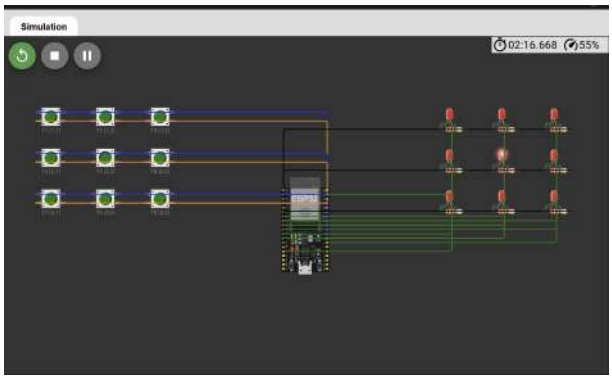


Pengujian T3 ke L3 (1,3).

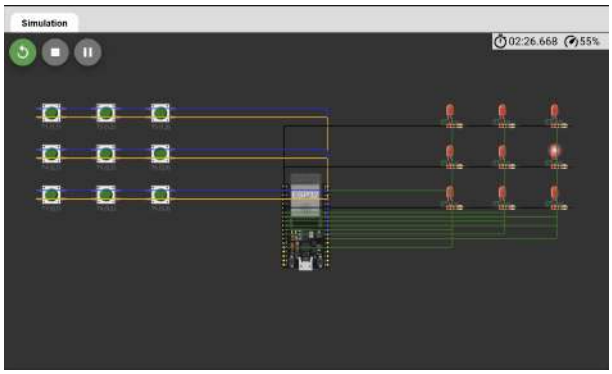


Pengujian T4 ke L4 (2,1).

Gambar 22: Hasil pengujian posisi (1,3) dan (2,1).

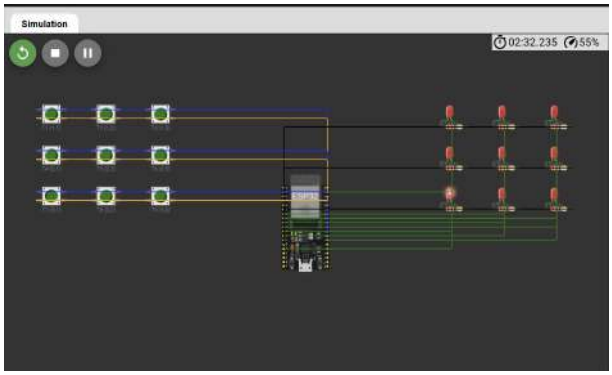


Pengujian T5 ke L5 (2,2).

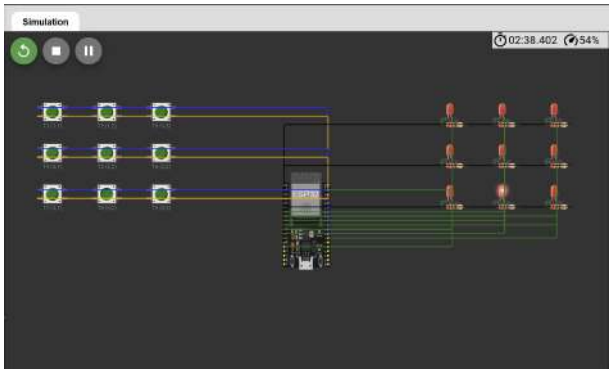


Pengujian T6 ke L6 (2,3).

Gambar 23: Hasil pengujian posisi (2,2) dan (2,3).

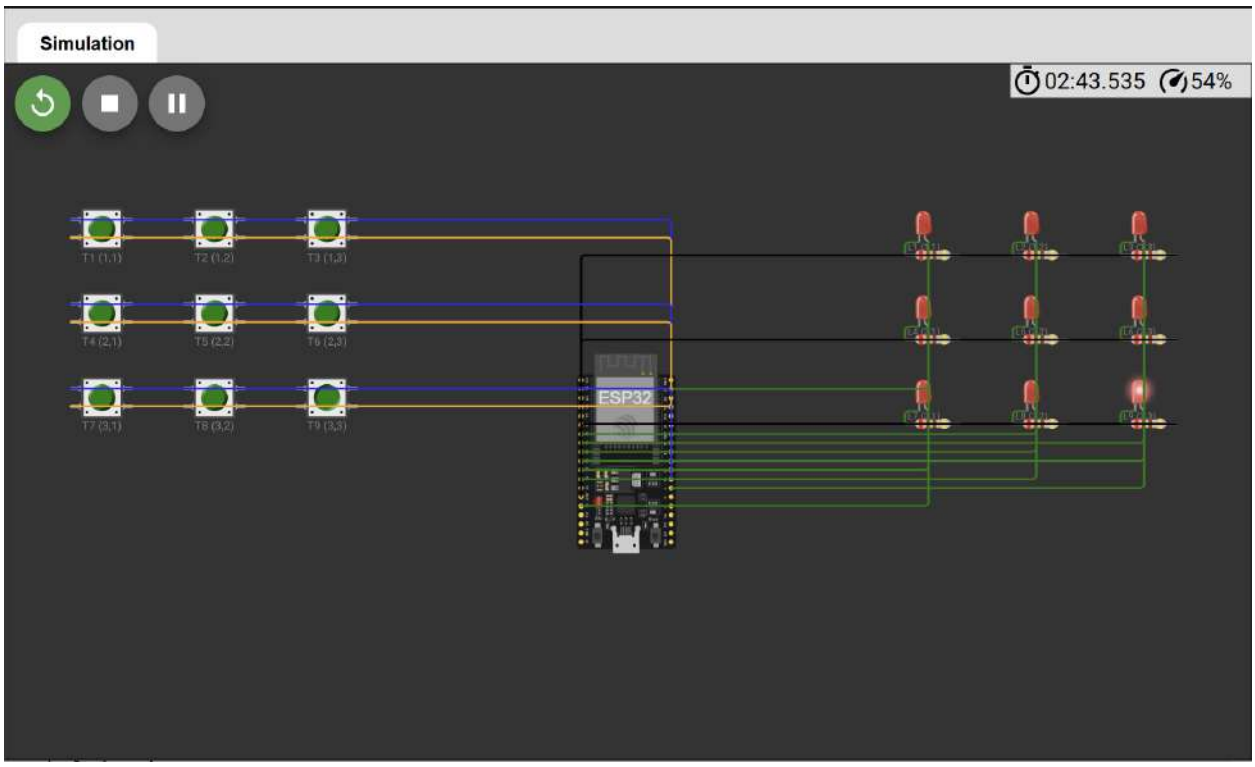


Pengujian T7 ke L7 (3,1).



Pengujian T8 ke L8 (3,2).

Gambar 24: Hasil pengujian posisi (3,1) dan (3,2).



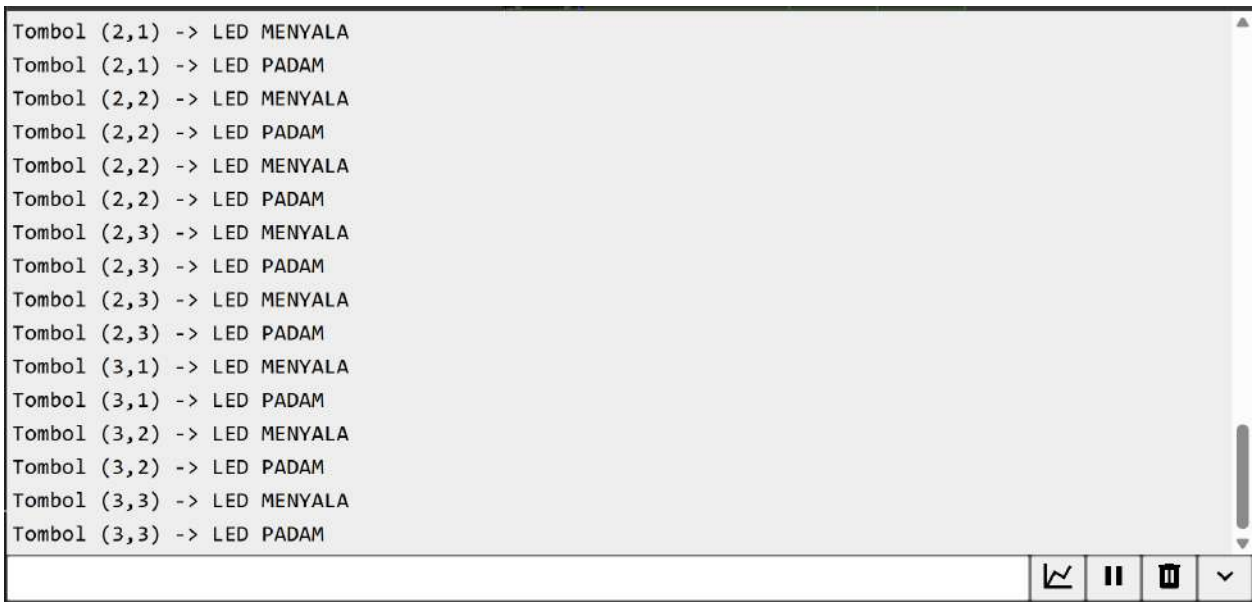
Gambar 25: Hasil pengujian T9 yang menyalakan L9 pada posisi (3,3).

3.4 Hasil Serial Monitor dan Analisis Respons Tombol

Serial Monitor Wokwi menampilkan koordinat tombol beserta perubahan keadaan LED. Pada cuplikan pertama terlihat pasangan pesan MENYALA dan PADAM untuk posisi (1,1), (1,2), (1,3), (2,1), (2,2), dan (2,3). Cuplikan kedua memuat pengujian pada baris kedua serta pasangan pesan untuk posisi (3,1), (3,2), dan (3,3). Kedua cuplikan secara bersama mencakup seluruh sembilan posisi tombol.

```
Tombol (1,1) -> LED MENYALA
Tombol (1,1) -> LED PADAM
Tombol (1,2) -> LED MENYALA
Tombol (1,2) -> LED PADAM
Tombol (1,2) -> LED MENYALA
Tombol (1,2) -> LED PADAM
Tombol (1,3) -> LED MENYALA
Tombol (1,3) -> LED PADAM
Tombol (2,1) -> LED MENYALA
Tombol (2,1) -> LED PADAM
Tombol (2,2) -> LED MENYALA
Tombol (2,2) -> LED PADAM
Tombol (2,2) -> LED MENYALA
Tombol (2,2) -> LED PADAM
Tombol (2,3) -> LED MENYALA
Tombol (2,3) -> LED PADAM
```

Gambar 26: Serial Monitor Wokwi: perubahan keadaan LED untuk tombol baris pertama dan kedua.



Gambar 27: Serial Monitor Wokwi: perubahan keadaan LED pada baris kedua dan ketiga.

Bentuk pesan yang muncul antara lain:

Listing 2: Contoh pesan perubahan keadaan pada Serial Monitor Wokwi.

```
Tombol (1,1) -> LED MENYALA
Tombol (1,1) -> LED PADAM
Tombol (2,2) -> LED MENYALA
Tombol (2,2) -> LED PADAM
Tombol (3,3) -> LED MENYALA
Tombol (3,3) -> LED PADAM
```

Pesan MENYALA menunjukkan program telah menerima keadaan tombol ditekan dan memberikan HIGH pada GPIO LED terkait. Pesan PADAM menunjukkan penerimaan keadaan tombol dilepas dan pengubahan GPIO LED menjadi LOW. Koordinat yang tercetak sesuai dengan pemetaan posisi pada array `ledPins`, sehingga hasil serial melengkapi pengamatan visual sembilan LED.

Pada kode, pencetakan pesan dan pembaruan LED berada di dalam kondisi perubahan `stablePressed` setelah validasi `DEBOUNCE_MS = 35`. Dengan demikian, log menampilkan perubahan keadaan stabil yang diterima program, bukan setiap pembacaan dalam siklus pemindaian. Pasangan pesan yang kembali muncul pada beberapa posisi menggambarkan beberapa siklus perubahan keadaan pada tombol yang sama. Urutan MENYALA lalu PADAM sesuai dengan rancangan LED yang mengikuti keadaan tombol stabil. Validasi diterapkan pada penekanan maupun pelepasan, sehingga kedua perubahan melewati mekanisme penyaringan yang sama.

Hasil ini menunjukkan hubungan yang konsisten antara pembacaan koordinat tombol, pembaruan LED tujuan, dan keluaran serial. Sembilan posisi menghasilkan pesan untuk keadaan aktif serta tidak aktif, sesuai fungsi kendali tombol–LED yang diminta pada tugas modul.

3.5 Jawaban Pertanyaan Analisis

3.5.1 Kebutuhan Debouncing pada Matriks Tombol

Getaran kontak merupakan sifat mekanik setiap tombol sehingga tetap dapat terjadi meskipun hanya satu tombol ditekan. Pemindaian berulang dapat menangkap beberapa perubahan sesaat sebagai

perubahan keadaan yang terpisah. Penyaring 35 ms pada rancangan ini menahan pembaruan sampai pembacaan stabil. Riwayat terpisah untuk setiap tombol memastikan perubahan pada satu posisi tidak mengganti waktu validasi posisi lain. Pembaruan LED dilakukan setelah validasi, sehingga keluaran mengikuti keadaan stabil dan tidak langsung mengikuti setiap transisi mentah.

3.5.2 Konfigurasi Pull-up dan Keadaan Masukan

GPIO19, GPIO21, dan GPIO22 menggunakan pull-up internal. Ketika tombol tidak ditekan, kolom berada pada HIGH. Saat tombol pada baris aktif ditekan, kolom terhubung ke baris LOW sehingga pembacaan menjadi LOW. Keadaan HIGH yang ditetapkan resistor mencegah kolom floating saat tombol terbuka. Kondisi ini diperlukan agar program dapat membedakan penekanan dari gangguan yang muncul pada masukan tanpa referensi logika. Baris yang tidak dipindai dikembalikan ke INPUT untuk mendukung pemilihan satu baris pada satu waktu.

3.5.3 Perbedaan Masukan Digital dan Analog

Tombol dibaca sebagai masukan digital dengan dua keadaan HIGH/LOW, sedangkan LDR dibaca melalui ADC sebagai kode yang mewakili tegangan. ADC 12 bit mempunyai 4096 tingkat, sehingga perubahan masukan dapat direpresentasikan bertahap, sebagaimana pembacaan LDR 723–4095 pada percobaan sebelumnya. Apabila tombol diganti dengan LDR, diperlukan pembagi tegangan dan pin ADC yang sesuai. Jalur pemindaian digital tidak langsung setara dengan rangkaian analog karena level baris ikut memengaruhi tegangan masukan. Prinsip pengukuran dan penyambungan sensor perlu mengikuti kebutuhan ADC.

3.6 Kode Program Tugas Modul

Listing 3: Program matriks tombol 3×3 dengan debounce 35 ms per tombol.

```
const byte ROWS = 3;
const byte COLS = 3;
const byte rowPins[ROWS] = {16, 17, 18};
const byte colPins[COLS] = {19, 21, 22};
const byte ledPins[ROWS][COLS] = {
  {23, 25, 26},
  {27, 32, 33},
  {13, 14, 4}
};

const unsigned long DEBOUNCE_MS = 35;

bool lastRaw[ROWS][COLS] = {};
bool stablePressed[ROWS][COLS] = {};
unsigned long lastChange[ROWS][COLS] = {};

void setup() {
  Serial.begin(115200);


  for (byte r = 0; r < ROWS; r++) {
    pinMode(rowPins[r], INPUT);
```

```
  }
  for (byte c = 0; c < COLS; c++) {
    pinMode(colPins[c], INPUT_PULLUP);
  }
  for (byte r = 0; r < ROWS; r++) {
    for (byte c = 0; c < COLS; c++) {
      pinMode(ledPins[r][c], OUTPUT);
      digitalWrite(ledPins[r][c], LOW);
    }
  }
  Serial.println("Matrix 3x3 siap. Debounce 35 ms/tombol.");
}

void updateButton(byte r, byte c, bool rawPressed) {
  unsigned long now = millis();

  if (rawPressed != lastRaw[r][c]) {
    lastRaw[r][c] = rawPressed;
    lastChange[r][c] = now;
  }

  if ((now - lastChange[r][c] >= DEBOUNCE_MS) &&
      (rawPressed != stablePressed[r][c])) {
    stablePressed[r][c] = rawPressed;
    digitalWrite(ledPins[r][c], rawPressed ? HIGH : LOW);

    Serial.print("Tombol (");
    Serial.print(r + 1);
    Serial.print(",");
    Serial.print(c + 1);
    Serial.print(") -> LED ");
    Serial.println(rawPressed ? "MENYALA" : "PADAM");
  }
}

void loop() {
  for (byte r = 0; r < ROWS; r++) {
    digitalWrite(rowPins[r], LOW);
    pinMode(rowPins[r], OUTPUT);
    delayMicroseconds(10);
    for (byte c = 0; c < COLS; c++) {
      bool rawPressed = (digitalRead(colPins[c]) == LOW);
      updateButton(r, c, rawPressed);
    }

    pinMode(rowPins[r], INPUT);
  }
  delay(1);
}
```

4 Kesimpulan

Praktikum Modul 1 memperlihatkan penggunaan mikrokontroler untuk keluaran digital, masukan digital, dan masukan analog. Pemrograman STM32 Blackpill melalui ST-Link berhasil dengan status SUCCESS dan Verified OK. LED PC13 dikendalikan menggunakan logika aktif LOW, sedangkan contoh ESP32 menggunakan GPIO2 melalui API ESP-IDF. Pergantian nyala dan padam menunjukkan fungsi GPIO sebagai keluaran digital, dengan periode rancangan 1 s berdasarkan dua jeda program masing-masing 500 ms.

Pembacaan tombol menunjukkan pull-up aktif LOW dan pull-down aktif HIGH. Pada pengujian internal, seluruh kombinasi keadaan kedua tombol menghasilkan logika yang sesuai. Pengujian debounce 50 ms menghasilkan counter mentah 17 dan counter terfilter 15 pada akhir cuplikan, yang memperlihatkan perbedaan antara pembacaan transisi langsung dan penerimaan keadaan stabil. Pembacaan LDR melalui GPIO34 menghasilkan kode ADC 723–4095 dengan estimasi tegangan program 0,58–3,30 V. Perubahan kode mencerminkan variasi masukan analog, sementara kode maksimum 4095 menunjukkan batas atas pembacaan ADC.

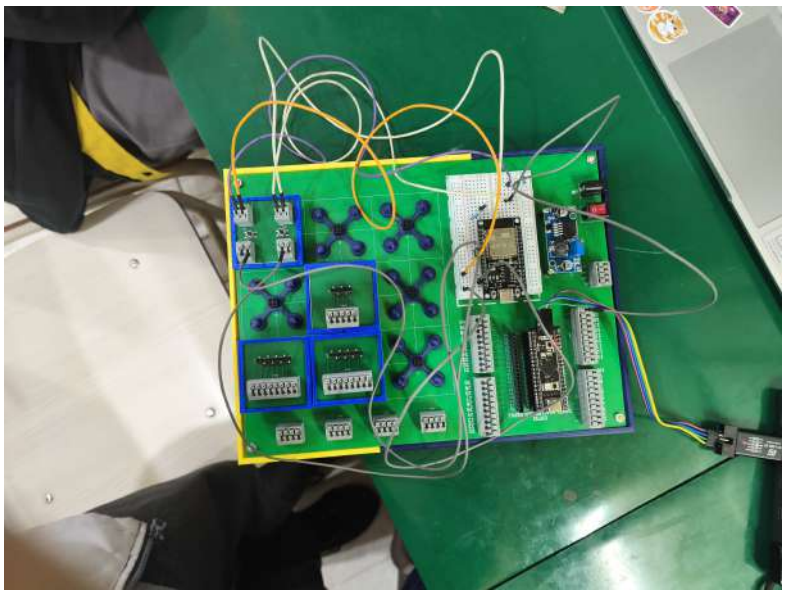
Tugas modul menerapkan pemindaian matriks tombol 3×3 dengan pull-up internal dan debounce 35 ms untuk setiap tombol. Pengujian sembilan posisi memperlihatkan LED pada posisi yang bersesuaian menyala secara bergantian. Serial Monitor menampilkan pesan MENYALA saat keadaan tekan diterima dan PADAM saat keadaan lepas diterima untuk seluruh sembilan koordinat, sehingga hasil visual dan keluaran program sejalan. Program menyimpan keadaan dan waktu perubahan setiap tombol secara terpisah, kemudian memperbarui LED setelah masukan stabil. Hasil praktikum dan simulasi menegaskan pentingnya pemetaan pin, kesesuaian logika rangkaian, serta validasi masukan sebelum digunakan untuk mengendalikan keluaran.

5 Lampiran

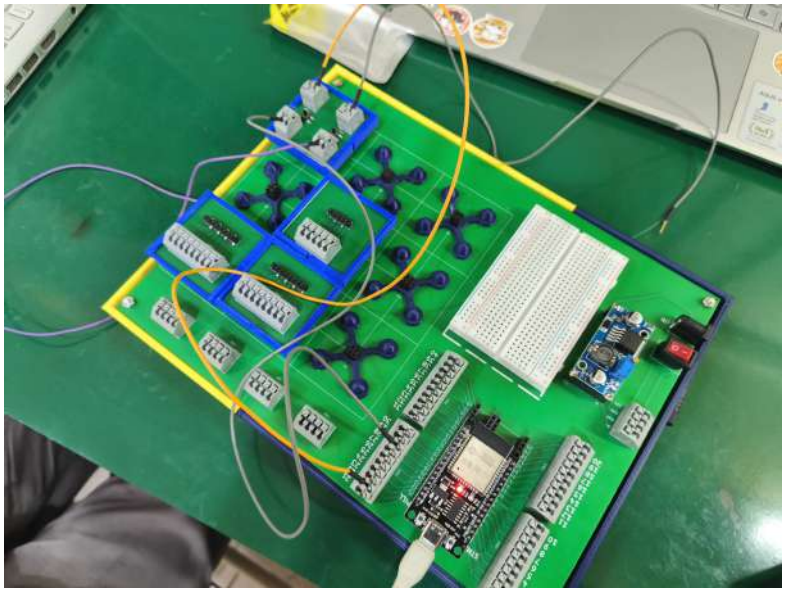
5.1 Dokumentasi saat praktikum



Gambar 28: Aktivitas penggunaan laptop dan trainer selama praktikum.

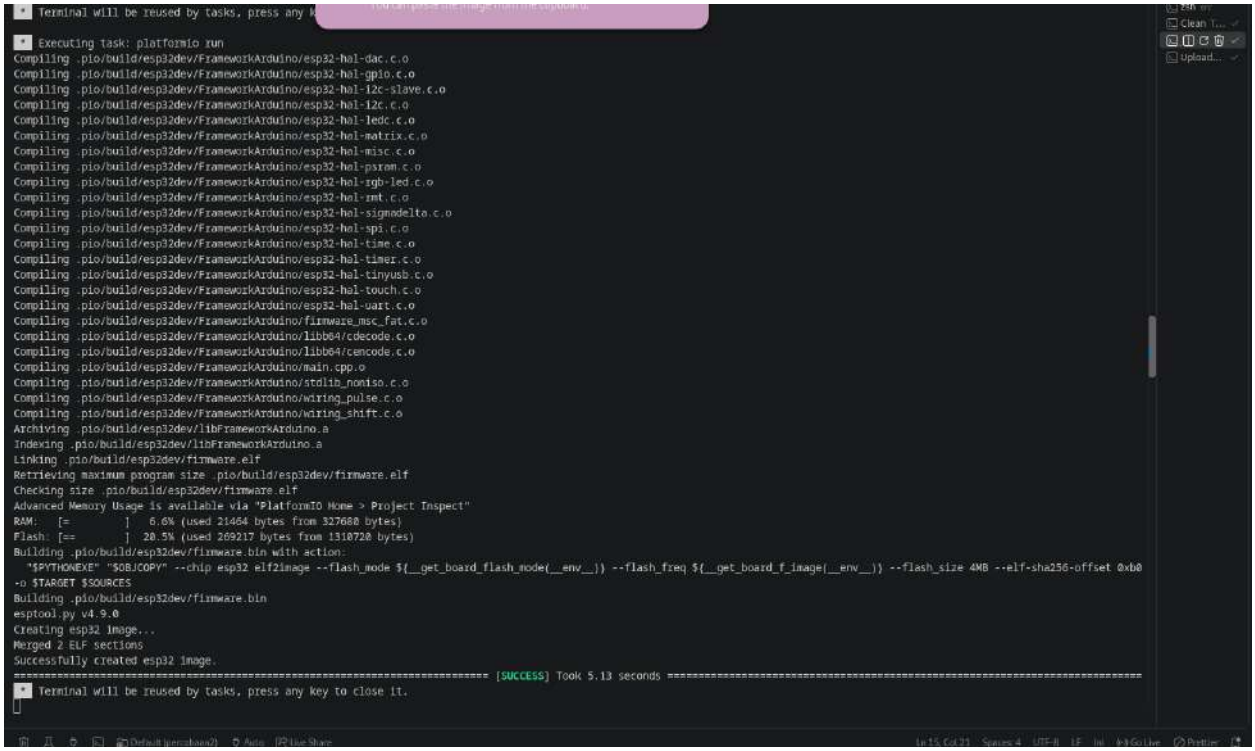


Gambar 29: Dokumentasi tambahan susunan trainer, breadboard, dan kabel pengujian tombol.

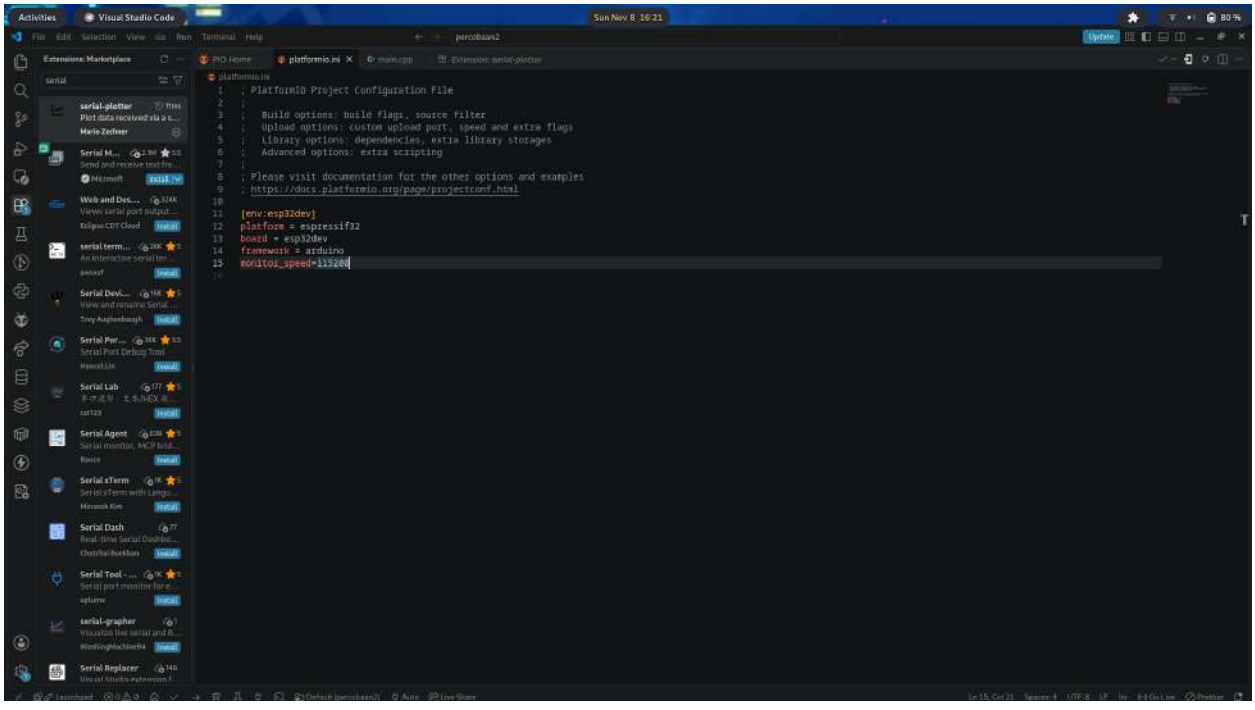


Gambar 30: Dokumentasi tambahan rangkaian tombol pada trainer.

5.2 Bukti Build dan Konfigurasi Pendukung



Gambar 31: Build ESP32 Arduino berstatus SUCCESS; RAM 6,6 persen dan Flash 20,5 persen pada log.



Gambar 32: Konfigurasi ESP32 Arduino dengan monitor serial 115200 baud pada tahap internal.